



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

Health Insurance Claim First Response Agent

PE6201 | Problem A | Group C-5 | Section C

Chen Baiyi Qian Ruoqi Fang Hioieng
Sood Vedansh Yang Yichen Zhang Qizhi

Video Demonstration Link:

<https://drive.google.com/file/d/1bqHumxLh23xIFu8GAbrsJzcZQlsBogXz/view?usp=sharing>

GitHub Link: <https://github.com/vedanshsood/PE6201-A2-Insurance-Claims-Agent-Group-5>

Drive Link (Project Staging):

https://drive.google.com/drive/folders/1m3gJrzo_FoHrOKr6ovJKNdoMY1yHAUw?usp=sharing

PART1 Why an Agent

1.1 Ladder Placement

For Problem A's health-insurance claim triage, our architecture is placed at Rung 7 (Agent) on Class 4's autonomy ladder. Lower rungs (Rungs 1–6: single calls, prompt chains, routing classifiers, parallel fans, orchestrators, and evaluator-optimisers) define enumerable, static workflows. They fail in claim adjudication because incoming claims feature non-linear, unpredictable branching across arbitrary line items, specific exclusions, and documentation criteria that hard-coded execution graphs cannot anticipate. However, an unconstrained Rung 7 agent introduces severe financial and regulatory risks, such as premature payout authorizations, hallucinated policies, or prompt injection via claimant narratives. To reconcile autonomy with corporate governance, we bound our ReAct loop with strict deterministic code-level guardrails (MAX_TURNS=8, TOKEN_BUDGET=60,000, action de-duplication) and enforce human-in-the-loop gating (AUTONOMY="confirm" on issue_decision_letter).

1.2 Workflow Test and Agentic Conditions

The claims environment satisfies both agentic criteria. First, execution trajectories cannot be scheduled beforehand: chasing pre-authorisations via get_preauthorisation is strictly conditional on the authoritative requires_preauthorisation boolean returned dynamically by check_coverage. Second, the loop receives machine-speed structured truth from deterministic local records rather than generating ungrounded assumptions. The four-question workflow test confirms that turn count varies dynamically with input—terminating in two turns on lapsed policies, but spanning four to six turns on complex multi-procedure claims.

1.3 Ground-Truth and Reliability Tests

Problem A satisfies the ground-truth speed test: five local JSON fixtures (members, policies, procedures, preauthorisations, and hospitals) answer in milliseconds, objectively falsifying invalid claims. Multi-turn execution subjects the agent to compounding reliability decay, where end-to-end success follows $P = s^T$, or $s = P^{1/T}$. In our live V2 evaluation using qwen/qwen-2.5-72b-instruct, the agent achieved a measured pass rate of $P = 88.33\%$ (53 passing trials out of 60) across an average horizon of $T = 5.47$ tool calls (328 total calls across the battery). This derives an empirical single-step reliability of: $s = (0.8833)^{(1 / 5.47)} \approx 0.9776$ (97.76%)

Because each failure escalates to a human assessor costing US\$7.60, reducing trajectory length directly minimizes Layer 2 fallback penalties.

1.4 What Good Looks Like

In production, a successful system requires: (1) factual grounding against official records without fabricated coverage; (2) deterministic compliance with the three canonical outcomes (approve_in_principle, request_document, or escalate); (3) executing the irreversible write (issue_decision_letter) at most once, strictly after full evidence resolution and confirmation; (4) unconditional escalation on hostile narrative injections or limit breaches; and (5) an operational cost per successful task strictly below the US\$7.60 human benchmark.

PART2 The Tool Layer

2.1 Tool Selection and Removal

Five read tools provide distinct evidence: `get_claim` retrieves claim facts and duplicate matches; `lookup_policy` returns member and policy records; `check_coverage` supplies procedure coverage and document requirements; `get_preauthorisation` retrieves records for assessing validity; and `get_hospital_status` returns hospital information. A sixth function, `issue_decision_letter`, is application-controlled and simulates issuance through an in-memory log. Exact identifiers reduce ambiguity from name-based lookups. Duplicate matching is incorporated into `get_claim`, and V2 adds document requirements to `check_coverage` instead of introducing another lookup. This limits the number of tool definitions presented to the model. No removal ablation establishes token savings. Application code should also recompute monetary totals; the live path currently checks only that supplied totals are numeric.

2.2 Tool Descriptor and V1→V2 Rewrite

V2 clarifies tool purposes, input identifiers, authoritative returns, routing conditions and relevant failure semantics. The central interface change adds `required_document` to `check_coverage`, alongside `procedure_code` and `coverage_status`. This poka-yoke makes procedure-specific document requirements directly observable rather than requiring the model to infer them. A second poka-yoke keeps `issue_decision_letter` application-controlled and unavailable to the live model, preventing direct execution of the gated write. The coverage descriptor instructs the model to compare `required_document` with submitted claim documents, while the preauthorisation descriptor requires records to be validated against the service date. Exact identifiers further reduce ambiguity in tool invocation.

Offline tokenisation of 67 coverage observations with the GPT-OSS-20B tokenizer gives means of 24.46 tokens in V1 and 31.97 in V2: an increase of 7.51 tokens (30.7%). These counts cover compactly serialised return content only, excluding message wrappers and accumulated context. For the same 42 cases, Chen Baiyi's V1 run achieved 33/42 decision-label matches (78.6%) using 316,526 recorded tokens. Selecting the first trial per case from Sood Vedansh's V2 battery gives 39/42 (92.9%) and 692,307 tokens. Thus, higher observed accuracy accompanied greater token use. Because return fields, descriptors, system instructions and empty-output handling changed together, this is a V1–V2 system comparison rather than an isolated descriptor ablation. Both runs use GPT-OSS-20B and the same case set, while retaining implementation and live-execution differences.

2.3 Dependency Rule

Calls may share a turn only when their inputs are known and neither needs the other's output. After `get_claim`, policy and hospital lookups can be grouped. Coverage waits for `policy_id`; preauthorisation waits for coverage to identify relevant procedures. Complete evidence is a workflow requirement, but the application's confirmation and structural checks do not independently establish evidence completeness. Coverage checks for different procedures can also share a turn once the policy identifier is available.

2.4 Sequential vs Parallel Tool Calling

The supplementary D2(c) summary reports a scripted comparison at a 32-turn cap: both modes execute 263 read calls and match 57/60 decision labels, while batching reduces backend turns from 323 to 239 (26.0%). These reported results concern offline scheduling. Calls within each batch still execute sequentially in Python; grouping reduces backend round trips. Live-model token savings and latency effects were not measured.

3. What the Evidence Showed

3.1 Evaluation Set

We evaluated the agent using team-designed ordinary cases together with the negative cases provided for the assignment. Rather than testing only straightforward approvals, our ordinary cases targeted boundary conditions and reasoning behavior, including policy-date and preauthorisation boundaries, multi-line resolution, annual-limit calculations, document handling, duplicate detection, and correct use of tool evidence. Some cases also tested false positives, such as distinguishing benign narrative language from hostile instructions. Separately, ten guardrail tests covered hostile-text protection, duplicate actions, autonomy gating, and budget control, with all ten passing.

3.2 Live Model Battery

In the controlled V1–V2 comparison, the V1 baseline achieved an 80% pass rate. V2 improved performance across all five tested models, but by different margins: GPT-OSS-20B reached 90.00%, Qwen 2.5 72B 88.33%, Gemini 2.5 Flash and DeepSeek Chat V3.1 both 85.00%, and GLM-4.5-Air 81.67%. The improvement therefore ranged from +1.67 to +10.00 percentage points. These results show that V2 improved overall performance, but the size of the improvement was not consistent across models.

3.3 Negative Cases and Model Differences

The negative cases revealed failures that were not visible in the ordinary cases. For GPT-OSS-20B V2, all six failed trials occurred within the negative set, which passed 21 of 27 trials (77.8%). Four were routing failures and two were system errors. In CLM-8925, the agent requested a document when escalation was required in two trials. More critically, in CLM-8952, it approved the claim in principle twice when the expected route was escalation. Together with the cross-model variation in Section 3.2, these results suggest that strong overall performance does not guarantee reliable handling of exceptional cases, and that the effectiveness of prompt and tool-interface refinements may vary across models.

4. What It Costs

We modelled the Problem A claim-first-response service at 8,000 claims per month using the Class 5 escalation model, not a retry model. A failed claim is escalated to a claims assessor, costing US\$38 per hour $\times$ 12 minutes / 60 = US\$7.60. Layer 1 uses live-battery input and output tokens $\times$ published list prices; provider-reported usage.cost is excluded because it may reflect discounts or caching. Layer 2 is $(1 - \text{pass rate}) \times \text{US\$7.60}$. Layer 3 is US\$0 only for this coursework baseline: no paid infrastructure, monitoring, or recurring service was evidenced.

openai/gpt-oss-20b is the deployment choice. It achieved the highest pass rate, 54/60 (90.0%), and the lowest list-price API cost. Its 60 trials used 908,279 input and 62,170 output tokens, giving Layer 1 of US\$0.000589 per task. Layer 2 is US\$0.760000; expected cost per successful task is therefore US\$0.760589 and the monthly baseline is US\$6,084.71 plus future Layer 3 cost. Qwen 2.5 72B Instruct, the next-best all-in alternative, costs US\$0.892420 per task and US\$7,139.36 per month at 88.3% pass rate. Reliability, not token price, drives the choice: expected human escalation contributes US\$6,080.00 monthly, versus US\$4.71 for model tokens.

Model	Pass rate (60 trials)	Layer 1 / task	Layer 2 / task	All-in / successful task	Monthly cost (8,000 claims)
openai/gpt-oss-20b	90.0%	US\$0.000589	US\$0.760000	US\$0.760589	US\$6,084.71
qwen/qwen-2.5-72b-instruct	88.3%	US\$0.005500	US\$0.886920	US\$0.892420	US\$7,139.36
google/gemini-2.5-flash	85.0%	US\$0.004775	US\$1.140000	US\$1.144775	US\$9,158.20
deepseek/deepseek-chat-v3.1	85.0%	US\$0.004058	US\$1.140000	US\$1.144058	US\$9,152.47
z-ai/glm-4.5-air	81.7%	US\$0.003162	US\$1.393080	US\$1.396242	US\$11,169.94

The cost ledger explains this result. We avoided a seventh tool that would add 142 prompt tokens every turn. Parallel calling reduced paired scripted runs from 323 to 239 turns (–26.0%) without changing 57/60 decision-label matches, and reduced serialised input by 29.2%. The V2 check_coverage observation grew from 24.46 to 31.97 bare-JSON tokens per call because it returns required_document. This small Layer 1 increase was justified: CLM-8901 changed from a V1 failure to 3/3 V2 passes. Success rate remains the dominant lever because it determines escalation exposure.

Cost lever	Measured evidence	Interpretation
Tool block size B	A rejected seventh tool would add 142 prompt tokens per turn.	Avoided recurring prefix cost by extending an existing tool instead.
Turn count T	323 $\rightarrow$ 239 scripted turns (–26.0%); 57/60 labels unchanged.	Parallel calls reduce round trips without changing observed correctness.
Observation size D	check_coverage: 24.46 $\rightarrow$ 31.97 bare-JSON tokens/call (+30.7%).	A bounded cost increase exposed the missing-document fact needed for correct routing.
Success rate	gpt-oss-20b: 54/60 = 90.0%.	Dominant lever: it determines expected US\$7.60 human escalation exposure.

Sensitivity confirms the risk: at 80%, 90%, and 100% success, gpt-oss-20b costs US\$12,164.71, US\$6,084.71, and US\$4.71 per month respectively. Its break-even success rate against Qwen is 88.27%; the observed 90.0% clears it by 1.73 percentage points. We ship an eight-turn cap and 60,000-token per-task ceiling. A per-user monthly limit remains an implementation requirement before deployment.

Part 5 The two failures

Two supplementary scripted experiments reproduced failures by deleting one component from the working implementation and then restoring it. Neither required a live API call.

First, under an identical scripted repetition stimulus for CLM-8850, disabling action de-duplication allowed eight identical `get_claim` calls before the eight-turn cap raised `GuardrailStop`. Logs recorded backend turns, executed calls, serialized input characters and stop reasons. Restoring the guard rejected the second attempt: turns fell from eight to two, executed reads from eight to one, and cumulative input characters from 100,404 to 19,659. This code-layer fix remembers tool names and arguments. Prompt instructions cannot enforce that memory; tool descriptions cannot monitor execution history. The step cap stopped repetition later; the token-budget check could not trigger because scripted usage remained zero.

After restoration, the normal scripted battery retained 57/60 decision-label matches, a median of four turns, a maximum of five, and zero cap stops. The guard contained the induced loop rather than producing a correct answer. Both reproductions purchased zero model tokens and incurred \$0 API cost; character counts are not token measurements.

Second, removing only `required_document` from `check_coverage` made CLM-8901 incorrectly return `approve_in_principle`. Restoring the field produced `request_document` for the missing itemised_bill. Both runs took four turns and four reads; recorded writes fell from one to zero. This belongs in the tool-interface layer because the authoritative observation omitted a required fact. Prompt wording cannot reconstruct missing policy evidence, while loop controls only constrain execution. Restoring the observation contract recovered the expected decision.

6 What we would not deploy

We would not deploy autonomous claim approval. The strongest reported model still misroutes negative cases, and decision-label scoring does not validate every amount or evidence trail. Confirmation authorises a write but does not prove its justification; a scripted probe produced an approval record from a confirmed final response without retrieval. Before deployment, we would require deterministic evidence and amount validation, durable audit records, a monthly user limit, and testing on unseen adversarial cases. Human review remains necessary for ambiguous claims.

We considered but did not build a second reviewer agent. It might catch unsupported approvals, missing documents or inconsistent totals, but shared models and evidence can reproduce correlated errors. It would add inference cost, latency, coordination and another evaluation obligation; these costs were not measured. We retained one model-directed controller and one write authority, prioritising deterministic checks and human confirmation before adding another model whose independent benefit remains unproven.